



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COMPILER DESIGN LAB MANUAL

**COURSE CODE : CSL 411
SEMESTER- VII**

PREPARED BY

Ms.Aiswarya M S

AP-CSE



MANGALAM COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Vision

To become centre of excellence in computing and research where future generations embrace technologies wholeheartedly and use its possibilities to make the world a better place to live.

Mission

Enlight the young talents to achieve academic excellence as well as professional competence by imparting state of the art knowledge in computing and to be admirable individuals with ethical values and appropriate skills.

Program Educational Objectives

PEO 1: Graduate will have strong foundation and profound knowledge in computing and allied engineering and be able to analyze the requirements of real world problems, design and develop innovative engineering solutions and maintain it effectively in the profession.

PEO 2: Graduate will adapt to technological advancements by engaging in higher studies, lifelong learning and research, there by contribute to computing profession.

PEO 3: Graduate will foster team spirit, leadership, communication, ethics and social values, which will lead to apply the knowledge of societal impacts of computing technologies.

Program Specific Outcomes

PSO 1: Apply the Principles of Computing in solving real world problems with sustainability.

PSO 2: Apply Futuristic technology in designing and developing hardware and software solutions.

MANGALAM COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
PROGRAM OUTCOMES & PROGRAM SPECIFIC OUTCOMES

PROGRAM OUTCOMES	
PO1	Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to solve complex engineering problems.
PO2	Problem Analysis : Identify, formulate, research literature, and analyze complex engineering problems to reach substantiated conclusions.
PO3	Design/Development of Solutions : Design solutions for complex engineering problems and design system components or processes.
PO4	Conduct Investigations of Complex Problems : Use research-based knowledge and methods to interpret data and provide valid conclusions.
PO5	Modern Tool Usage : Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools.
PO6	Engineer and Society: Apply reasoning to assess societal, health, safety, legal and cultural issues and the consequent
PO7	Environment and Sustainability: Understand the impact of professional engineering solutions in societal and environmental contexts.
PO11	Project Management and Finance: Demonstrate knowledge and understanding of engineering and management principles.
PO12	Life-long Learning : Recognize the need for, and have the preparation and ability to engage in independent and lifelong learning.

PROGRAM SPECIFIC OUTCOMES	
PSO1	Apply the knowledge of computing fundamentals, data structures, algorithms, and software engineering to design and implement software systems.
PSO2	Analyze and provide solutions to real-world problems through analytical thinking and innovation using suitable tools and technologies.
PSO3	Apply professional engineering practices and strategies for lifelong learning and career development in the field of computer science and engineering.

LIST OF EXPERIMENTS:

Sl.No	Experiments	Date
1	Introduction to Compiler	
2	Lexical Analyzer using C Language	
3	Introduction to Lex	
4	Count vowels and consonents using lex tool	
5	Display the number of lines, words and characters	
6 (a)	String replacement using lex	
6 (b)	To write a lex program to check if the input is string, interger or floating point number.	
7	Lexical analyzer using lex tool	
8	Familiarization of YAAC	
9	To generate a yacc specification to recognize a valid arithmetic expression that uses operators +, -, *, / and parenthesis.	
10	To Implement Top-Down Parser. Recursive descent parsing using C for the given grammar E -> E+T T T -> T*F F F -> (E) id	
11	To Implement Bottom-Up Parser (Shift Reduce Parser) Grammar: E->E+E E*E E/E a b	
12	To Implement Intermediate code generation for simple expressions.	
13	To Implement Code optimization Techniques-Constant folding	
14	To implement the back end of the compiler which takes three address code as input and produces 8086 assembly language code as output.	

Date:

EXPERIMENT – 1 INTRODUCTION TO COMPILER

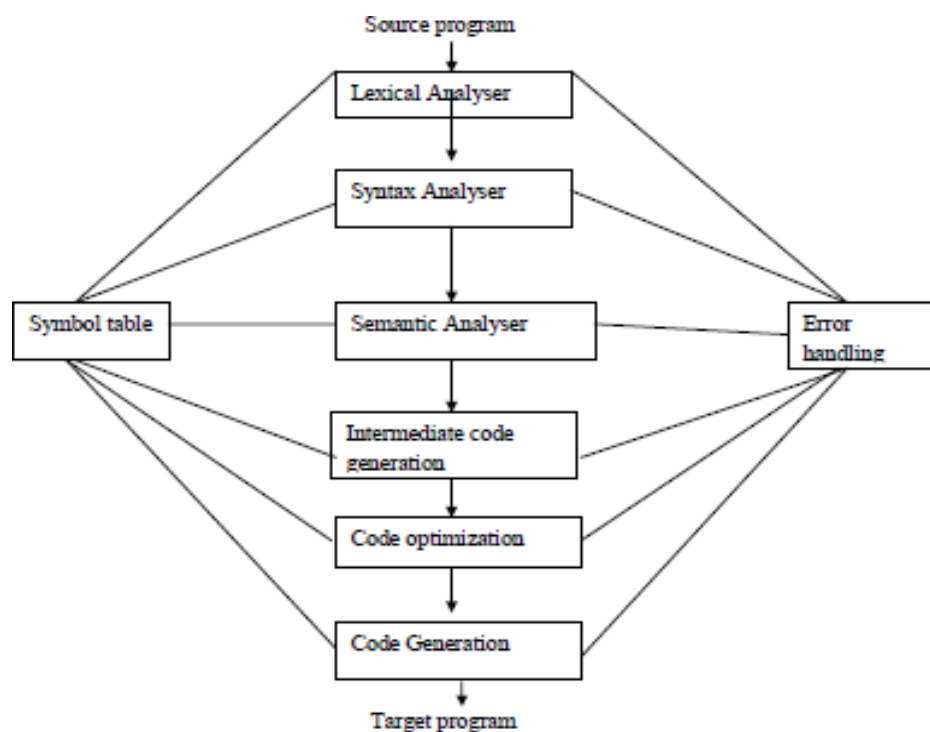
AIM:

To study the phases of compiler.

Introduction to Compiler

Compiler is a program which translates a program written in one language (the source language) to an equivalent program in other language (the target language). Usually the source language is a high level language like Java, C, FORTRAN etc. whereas the target language is machine code or "code" that a computer's processor understands. The source language is optimized for humans. It is more user-friendly, to some extent platform-independent. They are easier to read, write, and maintain and hence it is easy to avoid errors. Ultimately, programs written in a high-level language must be translated into machine language by a compiler. The target machine language is efficient for hardware but lacks readability.

Phases of compiler



Symbol Table Management:

- Symbol table is a data structure containing a record for each identifier, with fields for the attributes of the identifier.
- Record the identifier used in the source program and collect information about the identifier such as,
 - its type, (by semantic and intermediate code)
 - its scope, (by semantic and intermediate code)

- storage allocation, (by code generation)
- number of arguments and its type for procedure, the type returned

Error Detecting and Reporting:

- Each phase encounters errors.
- Lexical phase determine the input that do not form token.
- Syntax phase determine the token that violates the syntax rule.
- Semantic phase detects the constructs that have no meaning to operand.

The Analysis Phase:

- Involves: Lexical, Syntax and Semantic phase
- As translation progresses, the compiler's internal representation of the source program changes.

Lexical Analysis Phase:

- The lexical phase reads the characters in the source program and groups them into a stream of tokens in which each token represents a logically cohesive sequence of characters, such as, An identifier, A keyword, A punctuation character.
- The character sequence forming a token is called the lexeme for the token.

Syntax Analysis Phase:

- Syntax analysis imposes a hierarchical structure on the token stream. This hierarchical structure is called syntax tree.
- A syntax tree has an interior node is a record with a field for the operator and two fields containing pointers to the records for the left and right children.
- A leaf is a record with two or more fields, one to identify the token at the leaf, and the other to record information about the token.

Semantic Analysis Phase:

- This phase checks the source program for semantic errors and gathers type information for the subsequent code-generation phase.
- It uses the hierarchical structure determined by the syntax-analysis phase to identify the operators and operands of expressions and statements.
- An important component of semantic analysis is type checking.

Intermediate Code Generation:

- The syntax and semantic analysis generate a explicit intermediate representation of the source program.
- The intermediate representation should have two important properties:
 - It should be easy to produce,
 - And easy to translate into target program.
- Intermediate representation can have a variety of forms.
- One of the forms is: three address code; which is like the assembly language for a machine in which every location can act like a register.
- Three address code consists of a sequence of instructions, each of which has at most three operands.

Code Optimization:

- Code optimization phase attempts to improve the intermediate code, so that faster-running machine code will result.

Code Generation:

- The final phase of the compiler is the generation of target code, consisting normally of relocatable machine code or assembly code.
- Memory locations are selected for each of the variables used by the program.
- Then, the each intermediate instruction is translated into a sequence of machine instructions that perform the same task.

The input to a compiler may be produced by one or more preprocessors.

- The context in which the compiler typically operates are:
 - Preprocessors
 - Assemblers
 - Loader and Link-Editors

RESULT:

The experiment to study about the different phases of compiler have been done successfully.

Date:

EXPERIMENT – 2

LEXICAL ANALYZER USING C LANGUAGE

AIM:

To design and implement a lexical analyzer using c language.

Theoretical Background:

Lexical analyzer reads the input characters from the given program and produces the output as a sequence of tokens for syntax analysis phase. Example for tokens are keywords, identifiers, constants, operators, literals, punctuation, symbols such as parenthesis, commas and semicolons.

ALGORITHM:

1. Start
2. Inside main() function, define special symbols, operators, numbers, and other variables.
3. Open the input file in read mode using fopen().
4. If the fp==null, print error message,
5. Else read character by character,
6. Check if the character read matches with operators, special symbols, space or newline. If matches print it.
7. If the character read is alphabet, store it in buffer and check if it is keyword by calling the function isKeyword(), if yes print it as keyword.
8. Else print it as identifier.
9. Inside isKeyword(), declare the keywords as an array.
10. Check if the value in buffer is found in the array, if found set flag as 1 and return it to main().
11. Stop

PROGRAM:

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<ctype.h>

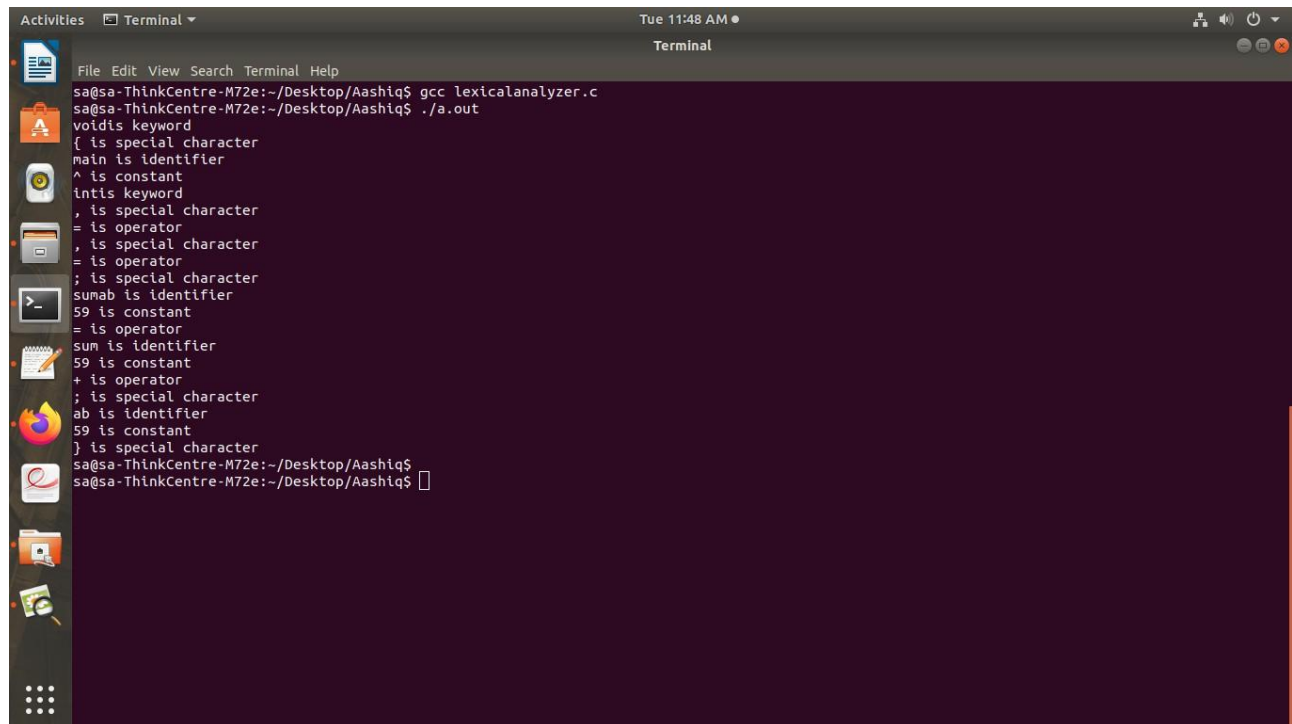
int isKeyword(char buffer[]){
char keywords[32][10] = {"auto", "break", "case", "char", "const", "continue", "default", "do",
"double", "else", "enum", "extern", "float", "for", "goto", "if", "int", "long", "register", "return",
"short", "signed", "sizeof", "static", "struct", "switch", "typedef", "union",
"unsigned", "void", "volatile", "while"};
int i, flag = 0;
for(i = 0; i < 32; ++i) {
if(strcmp(keywords[i], buffer) == 0) {
flag = 1;
break;
}
```

```
}
return flag;
}
int main(){
char ch, buffer[15], operators[] = "+-*/%=",specialch[]=".,[]{}",num[]="1234567890",buf[10];
FILE *fp;
int i,j=0,k=0;
fp = fopen("program.txt","r");
if(fp == NULL){
printf("error while opening the file\n");
exit(0);
}
while((ch = fgetc(fp)) != EOF)
{
for(i = 0; i < 6; ++i)
{
if(ch == operators[i])
{
printf("%c is operator\n", ch);
}
if(ch == specialch[i])
{
printf("%c is special character\n", ch);
}
}
if(isalpha(ch)){
buffer[j++] = ch;}
if(isdigit(ch)){
buf[k++] = ch;
}
else if((ch == ' ' || ch == '\n') && (j != 0)){
buffer[j] = '\0';
j = 0;
if(isKeyword(buffer) == 1)
printf("%sis keyword\n", buffer);
else{
printf("%s is identifier\n", buffer);
printf("%s is constant\n", buf);
}}
}
fclose(fp);
return 0;
}
```

INPUT

```
void main()
{
int sum,a=5,b=9;
sum= a+b;
}
```

OUTPUT



```
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$ gcc lexicalanalyzer.c
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$ ./a.out
void is keyword
{ is special character
main is identifier
^ is constant
int is keyword
, is special character
= is operator
= is operator
; is special character
sum is identifier
59 is constant
= is operator
sum is identifier
59 is constant
+ is operator
; is special character
ab is identifier
59 is constant
} is special character
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$
```

RESULT:

The program to implement lexical analyser using C have been executed successfully.

Date:

EXPERIMENT - 3 INTRODUCTION TO LEX

AIM:

To study about the implementation of lex tool.

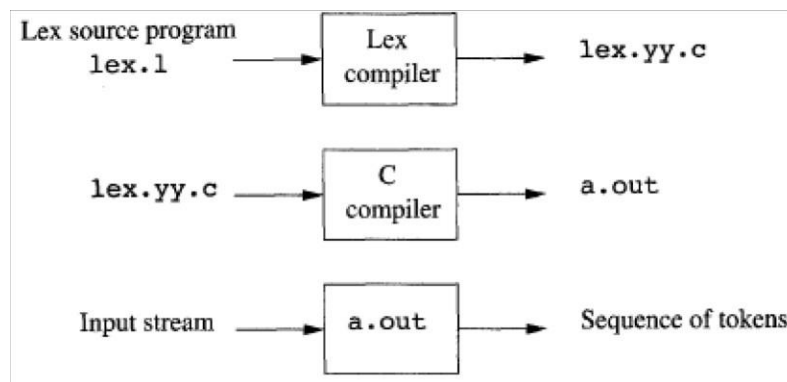
Lexers and Lexer Generators

Lex, is a tool that allows one to specify a lexical analyzer by specifying regular expressions to describe patterns for tokens. The input notation for the Lex tool is referred to as the Lex language and the tool itself is the Lex compiler. Behind the scenes, the Lex compiler transforms the input patterns into a transition diagram and generates code, in a file called `lex.yy.c`, that simulates this transition diagram. The mechanics of how this translation from regular expressions to transition diagrams occurs is the subject of the next sections; here we only learn the Lex language.

Use of Lex

Figure suggests how Lex is used. An input file, which we call `lex.l`, is written in the Lex language and describes the lexical analyzer to be generated. The Lex compiler transforms `lex.l` to a C program, in a file that is always named `lex.yy.c`. The latter file is compiled by the C compiler into a file called `a.out`, as always. The C-compiler output is a working lexical analyzer that can take a stream of input characters and produce a stream of tokens.

The normal use of the compiled C program, referred to as `a.out` in the Figure, is as a subroutine of the parser. It is a C function that returns an integer, which is a code for one of the possible token names. The attribute value, whether it be another numeric code, a pointer to the symbol table, or nothing, is placed in a global variable `yylval`, which is shared between the lexical analyzer and parser, thereby making it simple to return both the name and an attribute value of a token.



Creating a lexical analyzer with Lex

Structure of Lex Programs

A Lex program has the following form:

declarations

%%

translation rules

%%

auxiliary functions

The declarations section includes declarations of variables, manifest constants (identifiers declared to stand for a constant, e.g., the name of a token), and regular definitions.

The translation rules each have the form

Pattern { Action }

Each pattern is a regular expression, which may use the regular definitions of the declaration section. The actions are fragments of code, typically written in C, although many variants of Lex using other languages have been created.

The third section holds whatever additional functions are used in the actions. Alternatively, these functions can be compiled separately and loaded with the lexical analyzer.

The lexical analyzer created by Lex behaves in concert with the parser as follows. When called by the parser, the lexical analyzer begins reading its remaining input, one character at a time, until it finds the longest prefix of the input that matches one of the patterns P_i . It then executes the associated action A_i . Typically, A_i will return to the parser, but if it does not

(e.g., because P_i describes whitespace or comments), then the lexical analyzer proceeds to find additional lexemes, until one of the corresponding actions causes a return to the parser. The lexical analyzer returns a single value, the token name, to the parser, but uses the shared, integer variable `yylval` to pass additional information about the lexeme found, if needed.

RESULT:

Familiarization of LEX tool is done successfully.

Date:

EXPERIMENT - 4

COUNT VOWELS AND CONSONANTS USING LEX TOOL

AIM:

To write a lex program to find out total number of vowels and consonants from the given input sting.

ALGORITHM:

1. Start the program
2. In the declaration part, include the header file and initialise variables vow, cons.
3. In the transition rule section, include the pattern for vowels, and consonants.
4. In the main(), read the input.
5. Invoke yylex() function.
5. print the number of vowels and consonants in the given string.
6. Use the function yywrap() to handle the input reading.
7. Stop

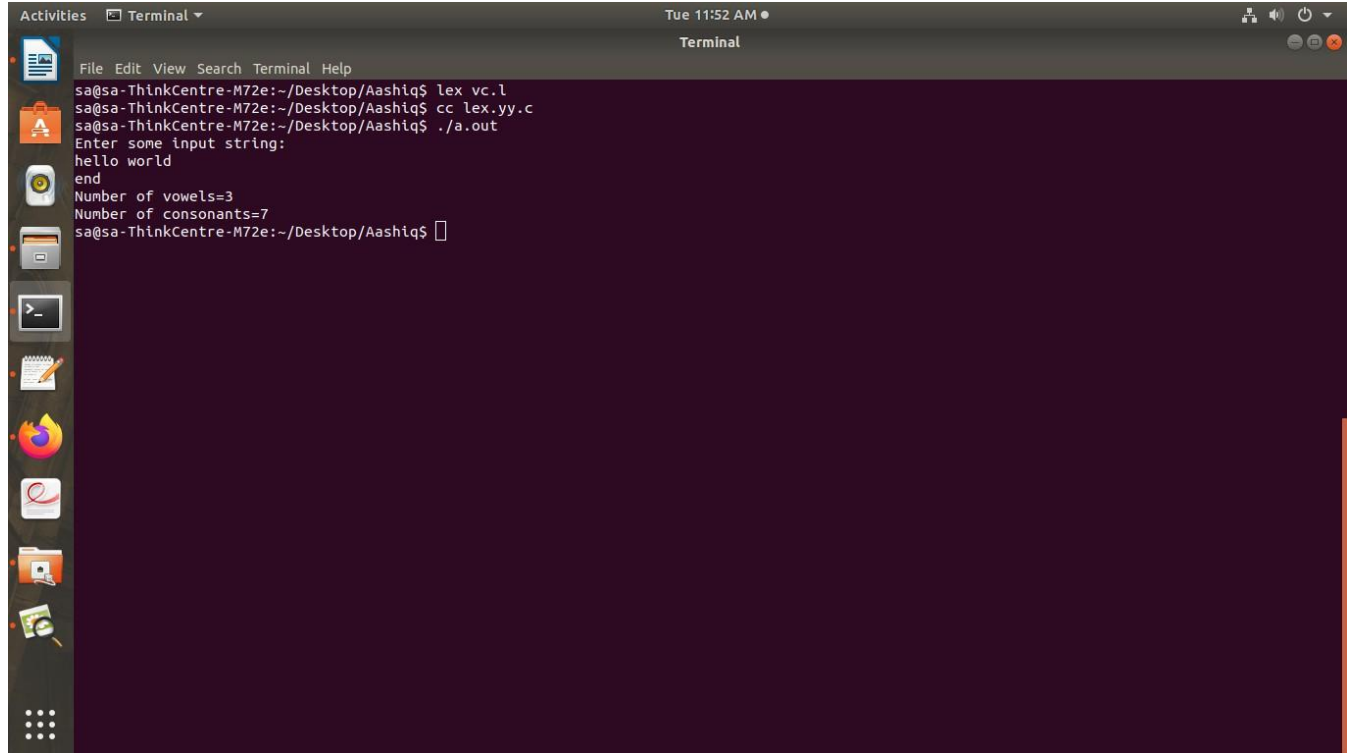
PROGRAM:

```
%{
#include<stdio.h>
int vow=0, con=0;
}%

%%
[ \t\n]+ ;
[aeiouAEIOU]+ {vow++;}
[^aeiouAEIOU] {con++;}
end return 0;
%%

int main( )
{
printf("Enter some input string:\n");
yylex();
printf("Number of vowels=%d\n",vow);
printf("Number of consonants=%d\n",con);
}

int yywrap( )
{
return 1;
}
```

OUTPUT:

```
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$ lex vc.l
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$ cc lex.yy.c
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$ ./a.out
Enter some input string:
hello world
end
Number of vowels=3
Number of consonants=7
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$
```

RESULT:

The program is compiled, executed and the output is verified.

Date:

EXPERIMENT - 5
DISPLAY THE NUMBER OF LINES, WORDS AND CHARACTERS USING LEX

AIM:

To write a lex program to display the number of lines, words and characters in an input text.

ALGORITHM:

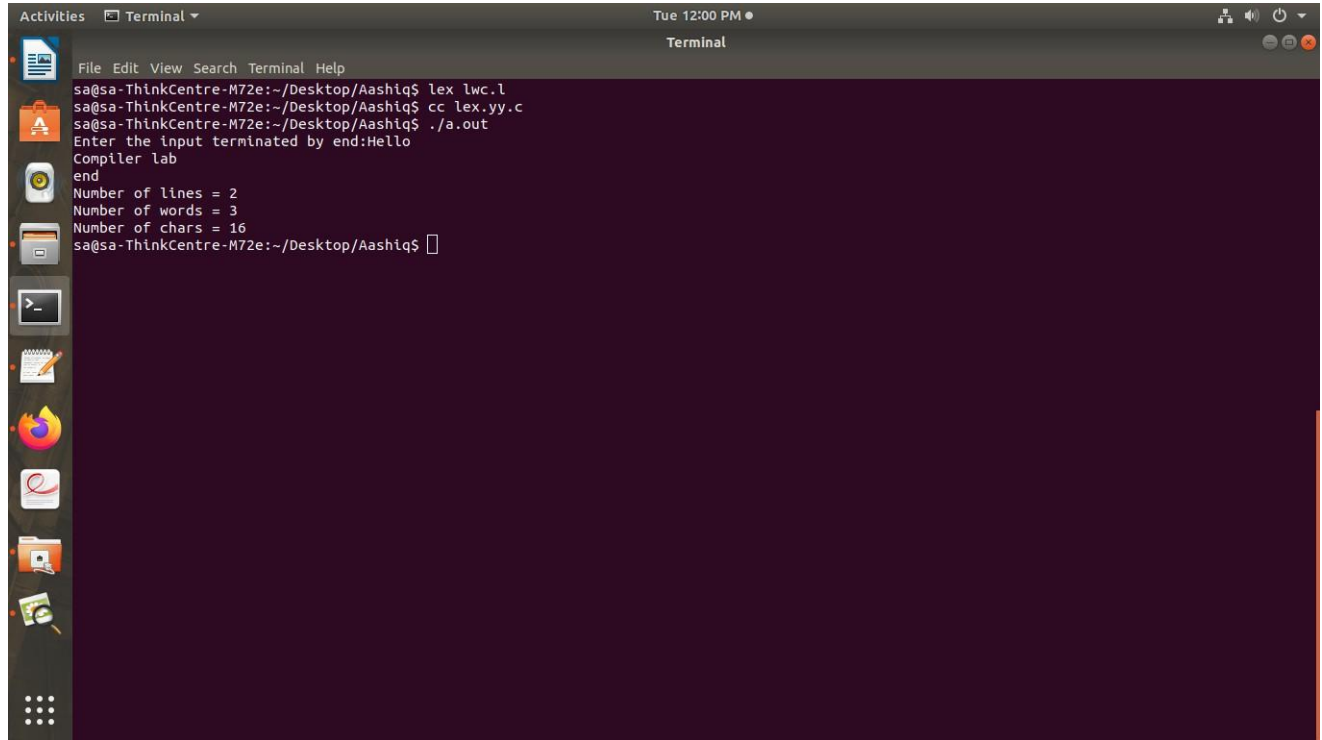
1. Include the header file, variables countl, countc, countw in the declaration section.
2. In the transition rules section, set the patterns for identifying lines, words and characters.
3. Use function yywrap() to handle input.
4. In the main function, read the input terminated by string 'end'
5. Display the number of characters, lines and words.
6. Stop

PROGRAM:

```
%{
#include<stdio.h>
int countl = 0;
int countc = 0;
int countw=0;
}%

%%
\n    ++countl;
[^\t\n ] ++countc;
.    ++countw;
end    return 0;
%%

/**/ User code section/**/
int yywrap() { }
int main(int argc, char **argv)
{
printf("Enter the input terminated by end:");
yylex();
printf("Number of lines = %d\nNumber of words = %d\nNumber of chars = %d\n", countl,
countw+countl, countc );
return 0;
}
```

OUTPUT:

```
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$ lex lwc.l
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$ cc lex.yy.c
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$ ./a.out
Enter the input terminated by end:Hello
Compiler lab
end
Number of lines = 2
Number of words = 3
Number of chars = 16
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$
```

RESULT:

The program is compiled, executed and the output is verified.

Date:

EXPERIMENT - 6 a

STRING REPLACEMENT USING LEX

AIM:

To write a lex program to count the number of scanf and printf in c program and replace them with readf and writef statement.

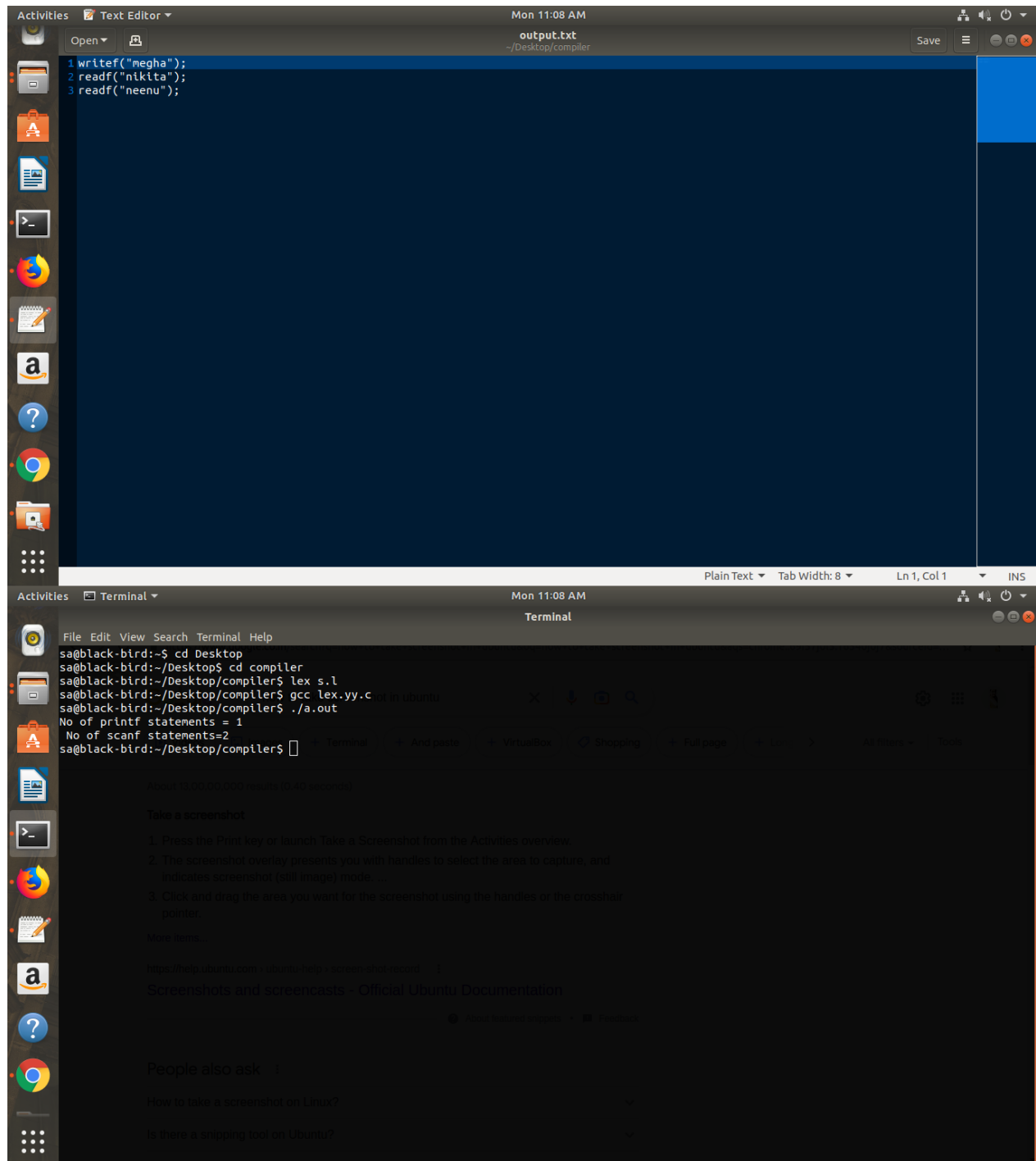
ALGORITHM:

1. In the declaration section, include the header files, and variables.
2. In the transition rule section, include the pattern to replace. Read the pattern 'printf' and replace it with 'writef'.
3. Read the pattern 'scanf' and replace it with 'readf'.
4. Include the yywrap() function.
5. In main(), open the file in read mode using yyin as input.
6. To write the output open a file in write mode into yyout.
7. Invoke the yylex() function.
8. Print the number of replaced statements.
9. Stop

PROGRAM:

```
%{
#include<stdio.h>
int pc=0, sc=0;
}%
%%
"printf" { fprintf(yyout,"writef"); pc++;}
"scanf" { fprintf(yyout,"readf"); sc++;}
%%
int yywrap()
{
return 1;
}
main()
{
yyin=fopen("input.txt","r");
yyout=fopen("output.txt","w");
yylex();
printf("No of printf statements = %d\n No of scanf statements=%d\n", pc, sc);
}
```

OUTPUT:



The screenshot displays a Linux desktop environment. The top window is a text editor titled 'output.txt' located at '~/.Desktop/compiler'. It contains the following C code:

```
1 writef("megha");
2 readf("nikita");
3 readf("neenu");
```

The bottom window is a terminal titled 'Terminal' at 'Mon 11:08 AM'. It shows the execution of the program:

```
sa@black-bird:~$ cd Desktop
sa@black-bird:~/Desktop$ cd compiler
sa@black-bird:~/Desktop/compiler$ lex s.l
sa@black-bird:~/Desktop/compiler$ gcc lex.yy.c -o in ubuntu
sa@black-bird:~/Desktop/compiler$ ./a.out
No of printf statements = 1
No of scanf statements=2
sa@black-bird:~/Desktop/compiler$
```

The terminal output indicates that the program executed successfully, printing the number of printf and scanf statements.

Result:

Thus, the program executed successfully.

Date:

EXPERIMENT - 6 b

STRING, INTEGER AND FLOATING-POINT NUMBERS USING LEX

AIM:

To write a lex program to check if the input is string, integer or floating point number.

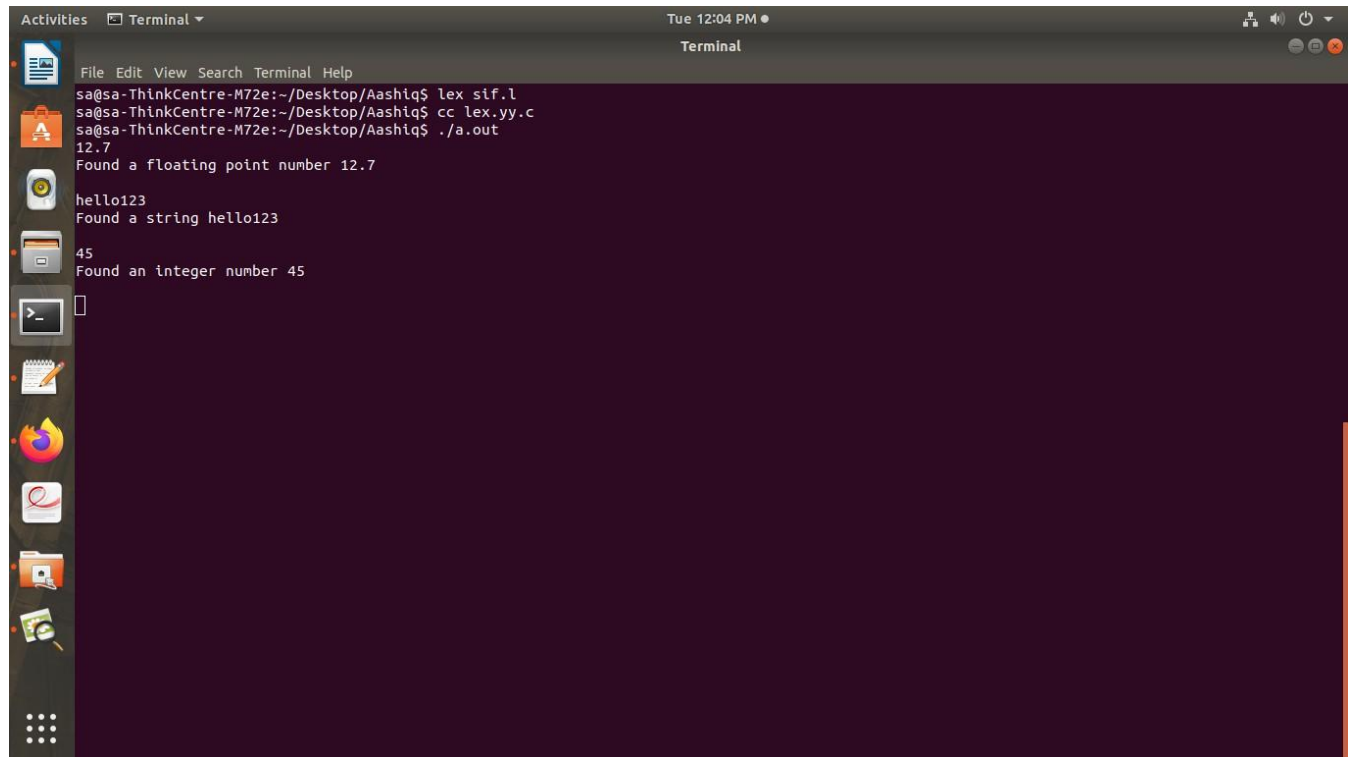
ALGORITHM:

1. In the declaration section, include the header files.
2. In the transition rule section, include the pattern to identify floating point number, integer and string.
3. Include the yywrap() function.
4. In main(), invoke the yylex() function.
5. Display the output.
6. Stop

PROGRAM:

```
%{  
#include<stdio.h>  
%}  
%%  
[\\t];  
[0-9]+\\. [0-9]+ {printf("Found a floating point number %s \\n",yytext);}   
[0-9]+ {printf("Found an integer number %s \\n",yytext);}   
[a-zA-Z0-9]+ {printf("Found a string %s \\n",yytext);}   
%%  
int yywrap() { return 1;}  
void main()  
{  
  yylex();  
}
```

OUTPUT:



```
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$ lex sif.l
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$ cc lex.yy.c
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$ ./a.out
12.7
Found a floating point number 12.7

hello123
Found a string hello123

45
Found an integer number 45


```

RESULT:

Thus, the program executed successfully.

Date:

EXPERIMENT-7

LEXICAL ANALYZER USING LEX TOOL

To implement a Lexical Analyzer for a given program using Lex Tool.

ALGORITHM

Token declaration Part

1. Declare token named Head as #include"<".*">"
2. Declare token named Key as
3. Declare token named Digit as [0-9]+
4. Declare token named Id as [a-z][A-Z0-9a-z0-9]*|_[a-z][A-Z0-9a-z0-9]*
5. Declare token named Op as "+","-","*","/",""

Translation Rule Part

1. If input lexeme matches with token {Head} then print it is a header file.
2. If input lexeme matches with token {Key} then print it is a keyword.
3. If input lexeme matches with token {Digit} then print it is a Digit.
4. If input lexeme matches with token {Id} then print it is an Identifier.
5. If input lexeme matches with token {Op} then print it is an operator.
6. If input lexeme matches with token ".", then skip other patterns.

Auxiliary Procedure Part

1. Start
2. Read the input file.
3. If input symbol is a keyword, print it is a keyword.
4. Check whether the current input is #include "<".*">", then print it is a header file.
5. Check whether the current input is digit, then print it is a digit.
6. Check whether the current input contains any identifier, then print it is an identifier.
7. Check whether the current input contains any operator, then print it is an operator.
8. Repeat the steps 2-8 until EOF.
9. Stop.

PROGRAM

```
%{
#include <stdio.h>
int integer=0,fp=0,kd=0,id=0,ar_op=0,rop=0,p=0,aop=0,ss=0;i=0,j=0,k=0,l,m,n;
char d[10][10],ide[10][10],ke[20][20];
}%

%%
[0-9]+ {integer++;strcpy(d[i],yytext);i++;}
[0-9]*"."[0-9]+ {fp++;}
("#include<stdio.h") {printf(" header file");}
("if"|"else"|"int"|"main"|"float"|"for"|"while") {kd++;strcpy(ke[j],yytext);j++;}
[_a-z]+[0-9a-z]* {id++;strcpy(ide[k],yytext);k++;}
```

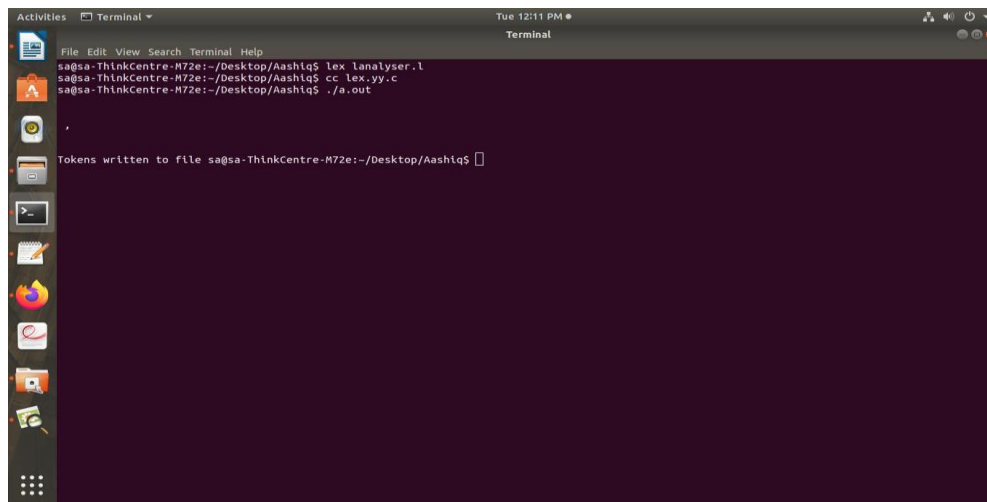
```
[ "+" | "-" | "*" | "?" ] {ar_op++;}  
[ "=" ] {aop++;}  
[ "!@#$%^&* _;'" ] {ss++;}  
[ "<" | ">" | ">=" | "<=" | "==" ] {rop++;}  
[ "{" | "}" | "(" | ")" ] {p++;}  
%%  
main()  
{  
yyin=fopen("f1.txt","r");  
yylex();  
yyout=fopen("f2.txt","w");  
fprintf(yyout,"Integers are \n");  
for(l=0;l<integer;l++)
```

```

fprintf(yyout,"%s\n",d[l]);
fprintf(yyout,"Keywords are \n");
for(m=0;m<kd;m++)
    fprintf(yyout,"%s\n",ke[m]);
fprintf(yyout,"Identifiers are \n");
for(n=0;n<id;n++)
    fprintf(yyout,"%s\n",ide[n]);
fprintf(yyout,"Integer =%d\n float=%d\n arithoperator=%d keyword=%d identifier=%d relational
operator=%d paranthesis=%d assignment operator=%d special
symbol=%d",integer,fp,ar_op,kd,id,rop,p,aop,ss);
}

```

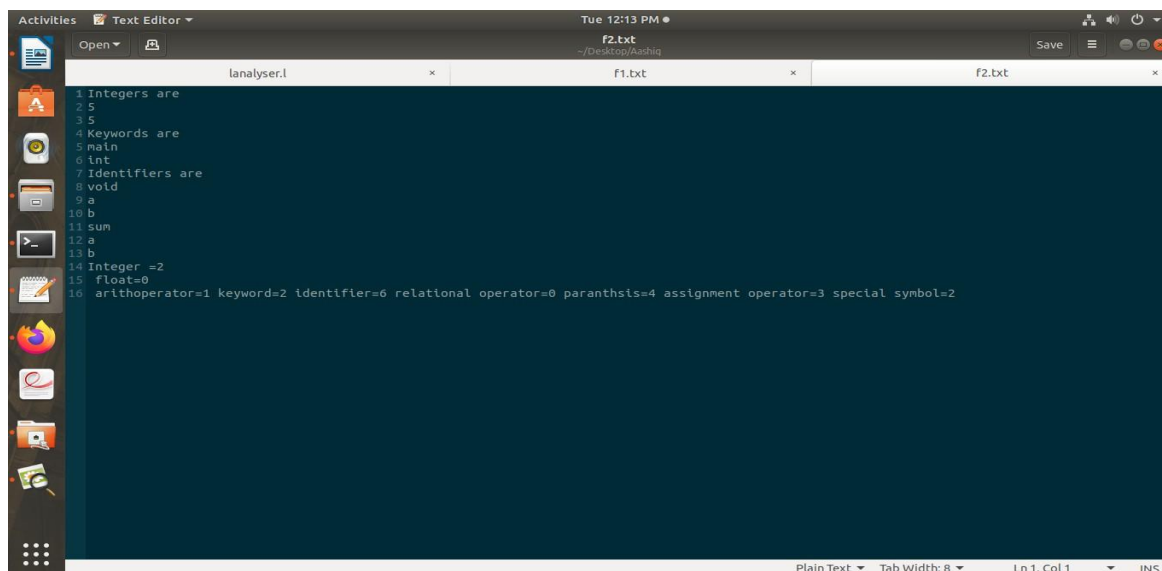
OUTPUT:



```

sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$ lex lanalyser.l
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$ cc lex.yy.c
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$ ./a.out
Tokens written to file sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$

```



```

1 Integers are
2 5
3 5
4 Keywords are
5 main
6 int
7 Identifiers are
8 void
9 a
10 b
11 sum
12 a
13 b
14 Integer =2
15 float=0
16 arithoperator=1 keyword=2 identifier=6 relational operator=0 paranthesis=4 assignment operator=3 special symbol=2

```

RESULT:

Thus, the program executed successfully.

Date:

EXPERIMENT - 8 **INTRODUCTION TO YACC**

AIM:

Familiarization of YACC tool

Yet Another Compiler Compiler(YACC)

The unix utility *yacc* (Yet Another Compiler Compiler) parses a stream of token, typically generated by *lex*, according to a user-specified grammar.

Structure of a yacc file

A *yacc* file looks much like a *lex* file:

```
...definitions...

%%
...rules...
%%

...code...
```

Definitions: All code between %{ and %} is copied to the beginning of the resulting C file.

Rules: A number of combinations of pattern and action: if the action is more than a single command it needs to be in braces.

Code: This can be very elaborate, but the main ingredient is the call to *yylex*, the lexical analyser. If the code segment is left out, a default main is used which only calls *yylex*.

Definitions section

There are three things that can go in the definitions section:

C code: Any code between %{ and %} is copied to the C file. This is typically used for defining file variables, and for prototypes of routines that are defined in the code segment.

Definitions: The definitions section of a *lex* file was concerned with characters; in *yacc* this is tokens. These token definitions are written to a .h file when *yacc* compiles this file.

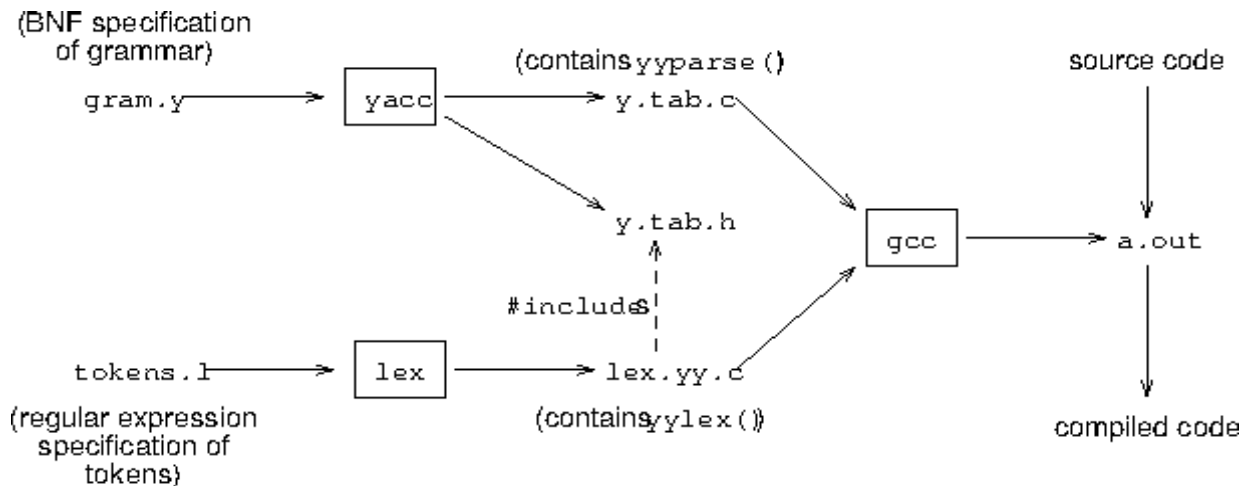
Associativity rules: These handle associativity and priority of operators.

Lex Yacc interaction

Conceptually, *lex* parses a file of characters and outputs a stream of tokens; *yacc* accepts a stream of tokens and parses it, performing actions as appropriate. In practice, they are more tightly coupled.

If the *lex* program is supplying a tokenizer, the *yacc* program will repeatedly call the *yylex* routine. The *lex* rules will probably function by calling return everytime they have parsed a token. *Lex* returns information in such a way that *yacc* can use it for parsing.

Marriage of lex and yacc



The shared header file of return codes

If *lex* is to return tokens that *yacc* will process, they have to agree on what tokens there are. This is done as follows.

- The *yacc* file will have token definitions
%token NUMBER
in the definitions section.
- When the *yacc* file is translated with *yacc -d*, a header file *y.tab.h* is created that has definitions like #define NUMBER 258

This file can then be included in both the *lex* and *yacc* program.

- The *lex* file can then call return NUMBER, and the *yacc* program can match on this token.

The return codes that are defined from %TOKEN definitions typically start at around 258, so that single characters can simply be returned as their integer value:

```

For eg : /* in the lex program */
         [0-9]+ {return NUMBER}
         [-+*/] {return *yytext}
/* in the yacc program */
sum : TERMS '+' TERM
  
```

Rules section

The rules section contains the grammar of the language you want to parse. This looks like

```

name1 :      THING something OTHERTHING {action}
         | othersomething THING {other action}

name2 : .....
  
```

This is the general form of context-free grammars, with a set of actions associated with each matching right-hand side. It is a good convention to keep non-terminals (names that can be expanded further) in lower case and terminals (the symbols that are finally matched) in upper case.

The terminal symbols get matched with return codes from the *lex* tokenizer. They are typically defined coming from %token definitions in the *yacc* program or character values.

Steps to Execute Lex Program:

```
lex <pgm name>
cc lex.yy.c -ll
./a.out
```

Steps to execute YACC program:

```
yacc -d <yacc_pgm name>
lex <lex_pgm_name>
cc y.tab.c lex.yy.c -ly -ll
./a.out
```

User code section

The minimal main program is

```
int main()
{
  yyparse();
  return 0;
}
```

Extensions to more ambitious programs should be self-evident. In addition to the main program, the code section will usually also contain subroutines, to be used either in the *yacc* or the *lex* program.

Regular Expression Basics

. : matches any single character except \n
 * : matches 0 or more instances of the preceding regular expression
 + : matches 1 or more instances of the preceding regular expression
 ? : matches 0 or 1 of the preceding regular expression
 | : matches the preceding or following regular expression
 [] : defines a character class
 () : groups enclosed regular expression into a new regular expression
 "...": matches everything within the " " literally

Special Functions

- yytext
 - where text matched most recently is stored
- yyleng
 - number of characters in text most recently matched
- yylval
 - associated value of current token
- yymore()
 - append next string matched to current contents of yytext
- yyless(n)
 - remove from yytext all but the first n characters
- unput(c)
 - return character c to input stream
- yywrap()
 - may be replaced by user
 - The yywrap method is called by the lexical analyser whenever it inputs an EOF as the first character when trying to match a regular expression

Files

y.output--Contains a readable description of the parsing tables and a report on conflicts generated by grammar ambiguities.

y.tab.c---- Contains an output file.

y.tab.h----- Contains definitions for token names.

yacc.tmp---- Temporary file.

yacc.debug --- Temporary file.

yacc.acts---- Temporary file.

RESULT:

Familiarization of YACC tool is done successfully.

Date:

EXPERIMENT – 9

YACC PROGRAM –ARITHMETIC EXPRESSION

AIM:

To generate a yacc specification to recognize a valid arithmetic expression that uses operators +, −, *, / and parenthesis.

ALGORITHM:

Lex File

1. In declaration section, include y.tab.h for making lexer read the input symbol so that it can send tokens to Yacc parser.
2. if input contains numbers, return NUM token to parser.
3. if input contains a identifier, return ID token to parser.
4. ignore \t tabspace.
5. if newline is encountered, it means end of expression, so return 0 to terminate lexer execution.
6. any other character is returned as it is.
7. yywrap() function is responsible for checking whether there are any more inputs. If no, then yywrap() tells program to end.

YACC File

1. In declaration section, include all header files required.
2. Define tokens which are getting returned from lexer.
3. Assign precedence and associativity of operators. first defined ones will have least preference. left means left associativity, and right means right associativity.
4. Define the grammar to examine the expression.
5. If input exactly satisfies this grammar, then we return to main() successfully, and print "Valid" in the main().
6. Else, yyerror gets called, and message "Invalid" gets printed.
7. Grammar is defined as, E -> e+e or e-e or e*e and so on including NUM and ID tokens.
8. This gets matched recursively as the input is read.
9. In main function, ask for expression. if calculated normally, display "valid expression".
10. Otherwise, yyerror() will be executed displaying "Invalid expression".
and display the evaluated answer in the grammar itself, when 'e' is valid.

PROGRAM:

Lex File : arvalidtex.l

```
%{
#include<stdio.h>
#include "y.tab.h"
extern int yylval;
}%
%%
[0-9]+ { yylval=atoi(yytext);
        return NUMBER;
```

```

    }
[a-zA-Z]+ {return ID;}
[\t] ;
[\n] return 0;
. return yytext[0];
%%
int yywrap()
{
return 1;
}

```

YACC file : arvalid.y

```

%{
#include<stdio.h>
int flag=0;

%}
%token NUMBER ID

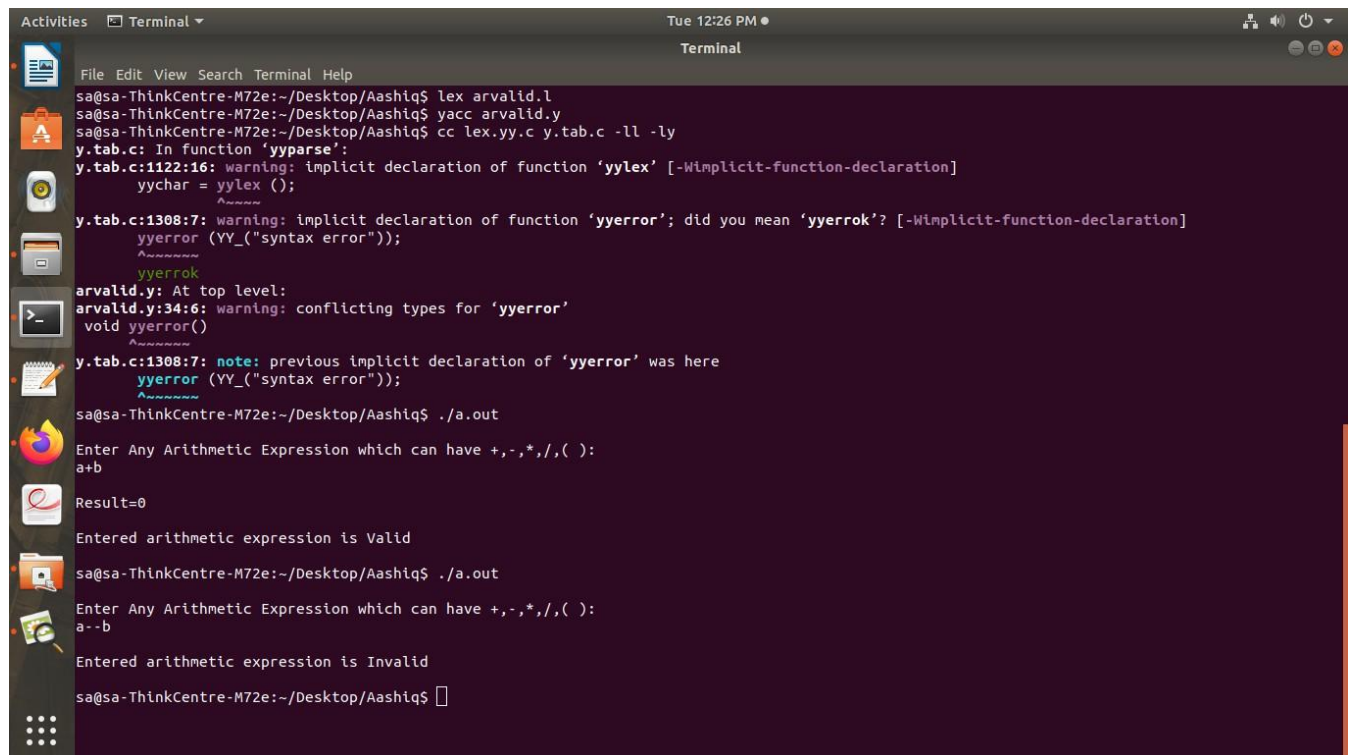
%left '+' '-'
%left '*' '/' '%'
%left '(' ')'
%%
ArithmeticExpression: E{
    printf("\nResult=%d\n",$$);
    return 0;
}
E:E+'E' {$$=$1+$3;}
|E-'E' {$$=$1-$3;}
|E'*E' {$$=$1*$3;}
|E/'E' {$$=$1/$3;}
|E'%E' {$$=$1%$3;}
|'('E')' {$$=$2;}
| NUMBER {$$=$1;}
| ID {$$=$1;}
;
%%

void main()
{
    printf("\nEnter Any Arithmetic Expression which can have +,-,*,/,(:\n");
    yyparse();
    if(flag==0)
        printf("\nEntered arithmetic expression is Valid\n\n");
}

```

```
}  
void yyerror()  
{  
    printf("\nEntered arithmetic expression is Invalid\n\n");  
    flag=1;  
}
```

OUTPUT:



```
Activities Terminal Tue 12:26 PM  
Terminal  
File Edit View Search Terminal Help  
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$ lex arvalid.l  
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$ yacc arvalid.y  
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$ cc lex.yy.c y.tab.c -ll -ly  
y.tab.c: In function 'yyparse':  
y.tab.c:1122:16: warning: implicit declaration of function 'yylex' [-Wimplicit-function-declaration]  
    yychar = yylex ();  
y.tab.c:1308:7: warning: implicit declaration of function 'yyerror'; did you mean 'yyerrok'? [-Wimplicit-function-declaration]  
    yyerror (YY_("syntax error"));  
yyerrok  
arvalid.y: At top level:  
arvalid.y:34:6: warning: conflicting types for 'yyerror'  
    void yyerror()  
y.tab.c:1308:7: note: previous implicit declaration of 'yyerror' was here  
    yyerror (YY_("syntax error"));  
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$ ./a.out  
Enter Any Arithmetic Expression which can have +,-,*,/, ( ):  
a+b  
Result=0  
Entered arithmetic expression is Valid  
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$ ./a.out  
Enter Any Arithmetic Expression which can have +,-,*,/, ( ):  
a-b  
Entered arithmetic expression is Invalid  
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$
```

RESULT:

Thus, the program executed successfully.

Date:

EXPERIMENT - 10

IMPLEMENT TOP-DOWN PARSER. (RECURSIVE DESCENT PARSER)

AIM:

To Implement Top-Down Parser.

Recursive descent parsing using C for the given grammar

$E \rightarrow E+T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

ALGORITHM:

First we have to avoid left recursion

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid id$

After eliminating Left recursion, we have to simply move from one character to next by checking whether it follow the grammar.

Algorithm:

Procedure E(): Begin

 T();

 EPRIME();

End:

Procedure EPRIME(): If input_symbol='+' then

 Begin

 ADVANCE();

 T();

 EPRIME();

 End;

Procedure T(): Begin

 F();

TPRIME();

End;

ProcedureTPRIME() : If input _symbol='*' then

Begin

ADVANCE();

F();

TPRIME();

End;

ProcedureF(): If input_symbol=id then

ADVANCE()

Else if input_symbol='(' then

Begin

ADVANCE();

E();

if input_symbol=')' then

ADVANCE();

Else ERROR();

End;

Else

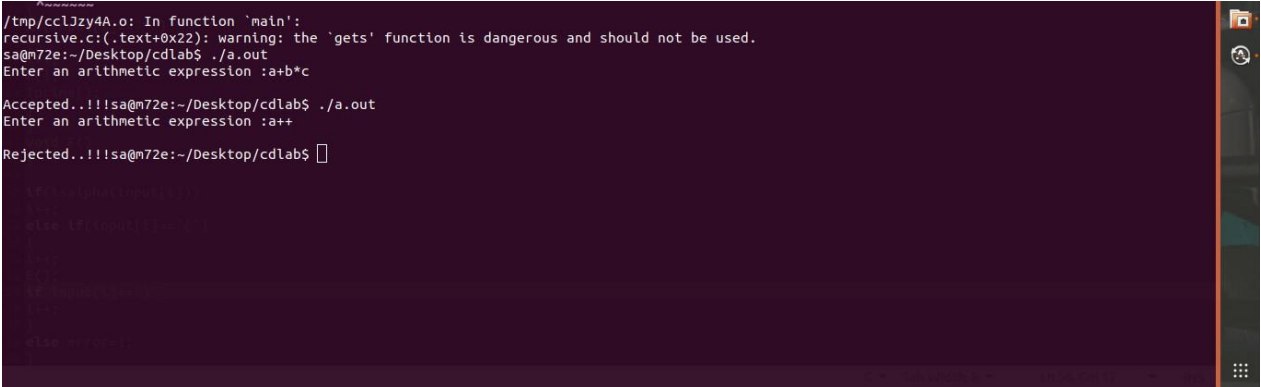
ERROR();

PROGRAM:

```
#include<stdio.h>
#include<string.h>
char input[10];
int i=0,error=0;
void E();
void T();
void Eprime();
void Tprime();
void F();
main()
{
```

```
printf("Enter an arithmetic expression :");
gets(input);
E();
if(strlen(input)==i&&error==0)
printf("\nAccepted...!!!");
else printf("\nRejected...!!!");
}
void E()
{
T();
Eprime();
}
void Eprime()
{
if(input[i]=='+')
{
i++;
T();
Eprime();
}
}
void T()
{
F();
Tprime();
}
void Tprime()
{
if(input[i]=='*')
{
i++;
F();
Tprime();
}
}
void F()
{

if(isalpha(input[i]))
i++;
else if(input[i]=='(')
{
i++;
E();
if(input[i]==')')
i++;
}
else error=1;
}
```

OUTPUT:

```
/tmp/cc1Jzy4A.o: In function 'main':
recursive.c:(.text+0x22): warning: the 'gets' function is dangerous and should not be used.
sa@m72e:~/Desktop/cdlab$ ./a.out
Enter an arithmetic expression :a+b*c

Accepted.!!!sa@m72e:~/Desktop/cdlab$ ./a.out
Enter an arithmetic expression :a++

Rejected.!!!sa@m72e:~/Desktop/cdlab$
```

RESULT:

Thus, the program to implement recursive descent parser has been executed successfully.

Date:

EXPERIMENT - 11

BOTTOM-UP PARSER (SHIFT REDUCE PARSER)

AIM

To Implement Bottom-Up Parser (Shift Reduce Parser)

Grammar:

$$E \rightarrow E + E \mid E^*E \mid E/E \mid a \mid b$$

ALGORITHM

1. Start with the sentence to be parsed as the initial sentential form
2. Until the sentential form is the start symbol do:
 - 2.1. Scan through the input until we recognise something that corresponds to the RHS of one of the production rules (this is called a handle)
 - 2.2. Apply a production rule in reverse; i.e., replace the RHS of the rule which appears in the sentential form with the LHS of the rule (an action known as a reduction)
3. A shift-reduce parser is most commonly implemented using a stack, where we proceed as follows:
 - 3.1 start with an empty stack
 - 3.2 a "shift" action corresponds to pushing the current input symbol onto the stack
 - 3.3 a "reduce" action occurs when we have a handle on top of the stack.
4. To perform the reduction, we pop the handle off the stack and replace it with the nonterminal on the LHS of the corresponding rule.
5. Stop

PROGRAM:

[illegible]

```

printf("\n$t\t%s$t\t\t--",ip_sym);
strcpy(act,"shift ");
temp[0]=ip_sym[ip_ptr];
temp[1]='\0';
strcat(act,temp);
len=strlen(ip_sym);
for(i=0;i<=len-1;i++)
{
    stack[str_ptr]=ip_sym[ip_ptr];
    stack[str_ptr+1]='\0';
    ip_sym[ip_ptr]=' ';
    ip_ptr++;
    printf("\n%s\t\t%s$t\t\t%s",stack,ip_sym,act);
    strcpy(act,"shift ");
    temp[0]=ip_sym[ip_ptr];
    temp[1]='\0';
    strcat(act,temp);
    check();
    str_ptr++;
}
check();
return 0;
}
int check()
{
    int flag=0;
    temp2[0]=stack[str_ptr];
    temp2[1]='\0';
    if(temp2[0]=='a' || temp2[0]=='b')
    {
        stack[str_ptr]='E';
        printf("\n%s\t\t%s$t\t\tE->%s",stack,ip_sym,temp2);
        flag=1;
    }
    if((!strcmp(temp2,"a")) || (!strcmp(temp2,"b")) || (!strcmp(temp2,"+")) || (!strcmp(temp2,"*")) || (!strcmp(temp2,"/")))
    {
        flag=1;
    }
    if((!strcmp(stack,"E+E")) || (!strcmp(stack,"E/E")) || (!strcmp(stack,"E*E")))
    {
        if(!strcmp(stack,"E+E"))
        {
            strcpy(stack,"E");
            printf("\n%s\t\t%s$t\t\tE->E+E",stack,ip_sym);
        }
        else if(!strcmp(stack,"E/E"))
        {
            strcpy(stack,"E");
            printf("\n%s\t\t%s$t\t\tE->E/E",stack,ip_sym);
        }
    }
}

```

```

else
{
strcpy(stack,"E");
printf("\n%s\t\t%s$\t\tE->E*E",stack,ip_sym);
}
flag=1;
str_ptr=0;
}
if(!strcmp(stack,"E")&&ip_ptr==len)
{
printf("\n%s\t\t%s$\t\tACCEPT",stack,ip_sym);
exit(0);
}
if(flag==0)
{
printf("\n%s\t\t%s$\t\tREJECT",stack,ip_sym);
exit(0);
}
return 0;
}

```

OUTPUT:

```

sa@sa-ThinkCentre-M72e: ~/Desktop
$ gcc SRP.c
$ ./a.out
SHIFT REDUCE PARSER

GRAMMER
E->E+E
E->E/E
E->E*E
E->a/b
Enter the input symbol: a+b/a

Stack Implementation Table
Stack      input symbol      action
-----
$          a+b/b*a$      --
$a         +b/b*a$      shift a
$E         +b/b*a$      E->a
$E+        b/b*a$      shift +
$E+b       /b*a$      shift b
$E+E       /b*a$      E->b
$E         /b*a$      E->E+E
$E/        b*a$      shift /
$E/b       *a$      shift b
$E/E       *a$      E->b
$E         *a$      E->E/E
$E*        a$      shift *
$E*a       $      shift a
$E*E       $      E->a
$E         $      E->E*E
sa@sa-ThinkCentre-M72e: ~/Desktop$

```

RESULT:

Thus, the program for performing shift reduce parsing for the grammar has been executed successfully.

Date:

EXPERIMENT - 12
INTERMEDIATE CODE GENERATION FOR SIMPLE EXPRESSIONS

AIM

To Implement Intermediate code generation for simple expressions.

ALGORITHM:

1. Declare the functions and variables globally.
2. Read the expression from user.
3. Push # on to the stack initially.
4. Scan all the symbols one by one from left to right in the given Infix Expression.
5. If the reading symbol is operand, then immediately append it to the Postfix Expression .
6. If the reading symbol is left parenthesis '(', then Push it onto the Stack.
7. If the reading symbol is right parenthesis ')', then Pop all the contents of the stack until the respective left parenthesis is popped and append each popped symbol to Postfix Expression.
8. If the reading symbol is operator (+, -, *, /), then Push it onto the Stack. However, first pop the operators which are already on the stack that have higher or equal precedence than the current operator and append them to the postfix . If open parenthesis is there on top of the stack then push the operator into the stack.
9. If the input is over, pop all the remaining symbols from the stack and append them to the postfix
10. Convert it into three address code format.
11. Display the result in tabular format.
12. Stop

PROGRAM:

```
#include<stdio.h>
#include<string.h>
#include<ctype.h>
int isp(char item);
void output(char item);
void push(char item);
char pop(void);
void quad(void);
char exp[20];
char res[20];
char a[20],opr[20],opd1[20],opd2[20],result[20];
```

```
int st[20],value[20];
int top=0,z=0,i=0,op1,op2,k,j,p,l;
char x,item;
void main()
{
printf("Enter the infix expression: ");
gets(exp);
l=strlen(exp);
push('#');
while((item=exp[i])!='\0')
{
if(isalpha(exp[i]))
output(item);
else if(item=='+'||item=='-'||item=='*'||item=='/'||item=='^')
push(item);
else if(item=='(')
push(item);
else if(item==')')
{
while((x=pop())!='(')
output(x);
}
else if(isp(x=pop())<isp(item))
{
push(x);
push(item);
}
else
{
output(x);
push(item);
}
i++;
}
while((x=pop())!='#')
output(x);
printf("Postfix expression: ");
puts(res);
quad();
}
int isp(char item)
{
if((item=='+'||item=='-'))
return(1);
```

```
else if((item=='*')||(item=='/'))
return(2);
else if((item=='^'))
return(3);
else
return(0);
}
void output(char item)
{
res[z++]=item;
}
void push(char item)
{
a[++top]=item; }
char pop(void)
{
item=a[top--];
return(item); }
void quad()
{
int i,x=0;
char m,n,p,temp,str1[5],str2[5];
printf("\noperator\ttop1\ttop2\tresult\n");
printf("-----");
for(i=0;i<l;i++)
{
if(isalnum(res[i]))
{
push(res[i]); }
else
{
if(isalpha(m=pop()))
{
str1[0]=m;
str1[1]='\0'; }
else
{
str1[0]='t';
str1[1]=m;
str1[2]='\0'; }
if(isalpha(n=pop()))
{
str2[0]=n;
str2[1]='\0'; }
```

```

else
{ str2[2]='\0'; }
x++;
printf("\n%c\t\t%s\t\t%d\n",res[i],str2,str1,x);
temp=x+'0';
push(temp);
} } }

```

OUTPUT:

```

sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$ gcc icg.c
icg.c: In function 'main':
icg.c:18:1: warning: implicit declaration of function 'gets'; did you mean 'fgets'? [-Wimplicit-function-declaration]
  gets(expr);
  ^~~~~
icg.c:18:1: note: 'gets' was declared here
icg.c:18:1: warning: the 'gets' function is dangerous and should not be used.
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$ ./a.out
Enter the infix expression: a+(b*c)-d
Postfix expression: abc*d-+

```

operator	op1	op2	result
*	b	c	t1
-	b	d	t2
+	a	t2	t3
+	a	t3	t4
+	a	t4	t5

```

sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$

```

RESULT:

Thus, the program has been executed successfully.

Date:

EXPERIMENT - 13
CODE OPTIMIZATION TECHNIQUES- (CONSTANT FOLDING)

AIM

To Implement Code optimization Techniques-Constant folding

ALGORITHM:

1. Start the program.
2. Read an expression from the user.
3. Check whether both side of an operator is a digit.
4. If true do the operation and save as t(eg. a+b+2+3,t=2+3;t=5.)
5. Update expression with constant as t.(new expression:a+b+5)
6. Display new Expression
7. Stop

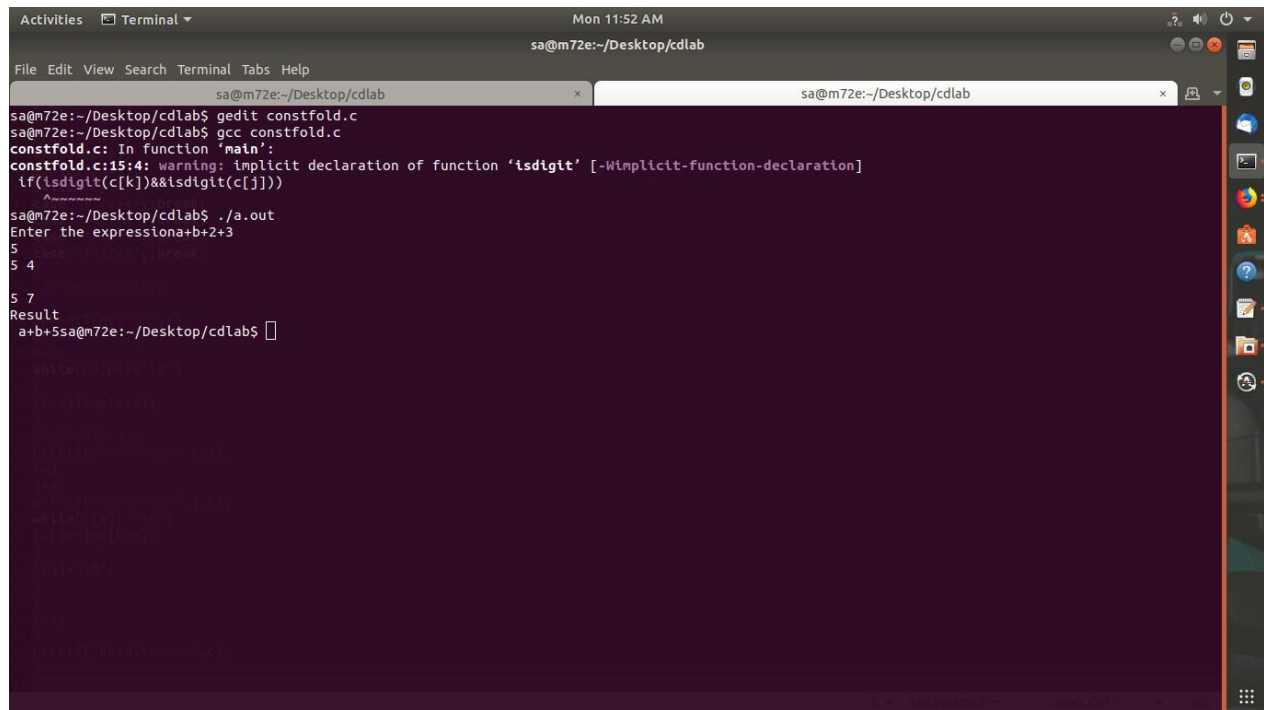
PROGRAM:

```
#include<stdio.h>
#include <stdlib.h>
void main()
{
char c[100],pre[10],post[10],ne[10];
int i,j,k,l,m,x,y,z,n;
printf("Enter the expression");
scanf("%s",c);
i=0;
while(c[i]!='\0'){
if(c[i]=='+'||c[i]=='-'||c[i]=='*'||c[i]=='/')
{
j=i-1;
k=i+1;
if(isdigit(c[k])&&isdigit(c[j]))
{
l=0;m=0;
while(isdigit(c[k]))
{
post[l++]=c[k++];
}
post[l]='\0';
while(isdigit(c[j]))
{
pre[m++]=c[j--];
}
pre[m]='\0';
```

```
x=atoi(pre);
y=atoi(post);
switch(c[i]){
case '+':z=x+y;break;
case '-':z=x-y;break;
case '*':z=x*y;break;
case '/':z=x/y;break;
}
//itoa(z,ne,10);

sprintf(ne,"%d",z);
printf("%s",ne);
n=0;
while(ne[n]!='\0')
{
c[++j]=ne[n++];
}
//c[++j]='z';
printf("\n%d %d\n",z,j);
i=j;
j++;
printf("\n%d %d\n",j,k);
while(c[k]!='\0')
{c[j++]=c[k++];
}
c[j]='\0';
}
}
i++;
}
printf("Result\n %s",c);
}
```

OUTPUT:



```
sa@m72e:~/Desktop/cdlab$ gcc constfold.c
constfold.c: In function 'main':
constfold.c:15:4: warning: implicit declaration of function 'isdigit' [-Wimplicit-function-declaration]
    if(isdigit(c[k])&&isdigit(c[j]))
    ~~~~~^
sa@m72e:~/Desktop/cdlab$ ./a.out
Enter the expression a+b+2+3
5
5 4
5 7
Result
a+b+5sa@m72e:~/Desktop/cdlab$
```

RESULT:

Thus, the program to perform code optimization has been executed successfully.

Date:

EXPERIMENT - 14
IMPLEMENT THE BACK END OF THE COMPILER

AIM

To implement the back end of the compiler which takes three address code as input and produces 8086 assembly language code as output.

ALGORITHM:

1. Read the intermediate code from the user.
2. Check for operatorstr[3], if + encountered copy ADD to opr.
3. Similarly for - , * , / copy SUB, MUL, DIV to opr respectively.
4. Use MOV with str[2].
5. Print the result.
6. Stop

PROGRAM:

```
#include<stdio.h>
#include<string.h>
void main() {
char icode[10][30],str[20],opr[10];
int i= 0;
printf("\n Enter the set of intermediate code (terminated by exit):\n");
do
{
scanf("%s",icode[i]);
} while(strcmp(icode[i++],"exit") != 0);
printf(" \n target code generation");
printf("\n*****");
i= 0;
do {
strcpy(str,icode[i]);
switch (str[3]) {
case '+': strcpy(opr,"ADD "); break;
case '-':
strcpy(opr,"SUB "); break;
case '*':
strcpy(opr,"MUL "); break;
case '/':
strcpy(opr,"DIV "); break;
```

```
}  
printf("\n \tMov %c,R%d",str[2],i);  
printf(" \n \t%s%c,R%d",opr,str[4],i); printf(" \n \tMov R%d,%c",i,str[0]);  
} while (strcmp(icode[++i],"exit") != 0);  
}
```

OUTPUT:



```
Terminal  
File Edit View Search Terminal Help  
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$ gcc backend.c  
sa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$ ./a.out  
  
Enter the set of intermediate code (terminated by exit):  
a=b+c  
d=a*a  
exit  
  
target code generation  
*****  
      Mov b,R0  
      ADD c,R0  
      Mov R0,a  
      Mov a,R1  
      MUL a,R1  
      Mov R1,dsa@sa-ThinkCentre-M72e:~/Desktop/Aashiq$
```

RESULT:

Thus, the program to implement back end of a compiler has been executed successfully.